

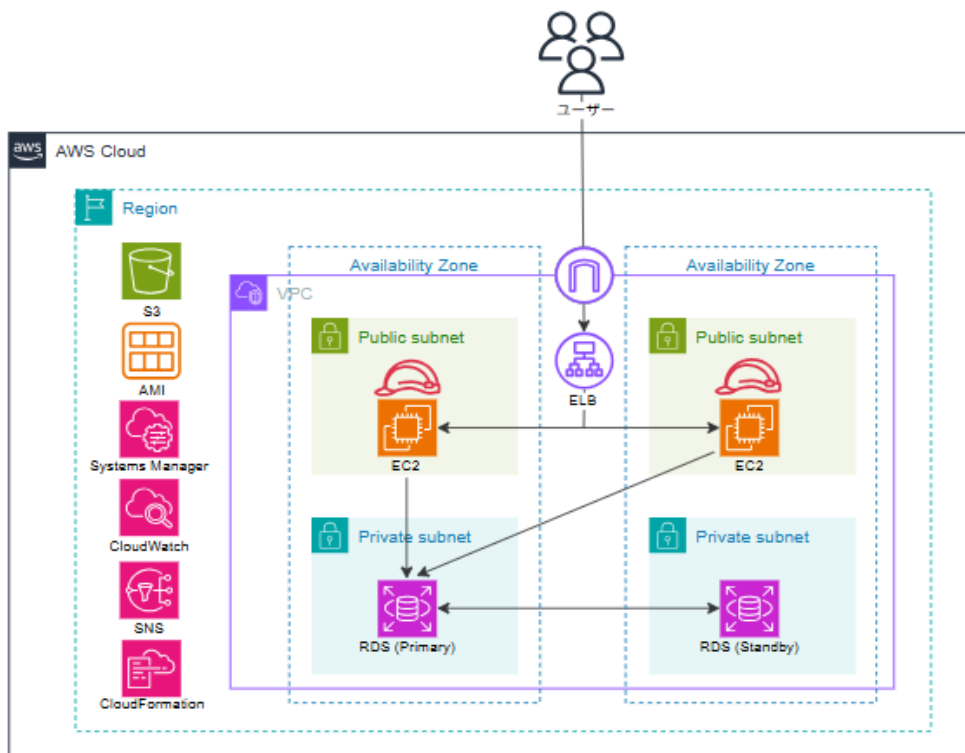
高可用なWebサイト構築手順書

概要

初級編では、以下の内容を実施します。

- VPC、EC2、RDS、S3、ELBを利用して、
可用性の高いブログサイト（WordPress）を構築する
- EC2 Instance Connect / Systems Managerを使用した EC2 インスタンスへの
コマンド実行を行い、
CloudWatchやCloudWatchLogsを利用したモニタリングを行う
- CloudFormation、Terraformを利用して、シンプルな構成のコードによる構築
を行う

最終的には次の構成図で示すようなアプリケーション環境を構築します。



演習の流れ

環境の構築は次のステップに分けて取り組んでください。

- タスク1. VPCリソースの作成
- タスク2. EC2とRDSの作成
- タスク3. S3の作成とIAMロールの利用
- タスク4. ELBの作成と環境の冗長化
- タスク5. SystemsManagerとCloudWatchの利用
- タスク6. IaCによるリソースの作成

本資料に関する注意点

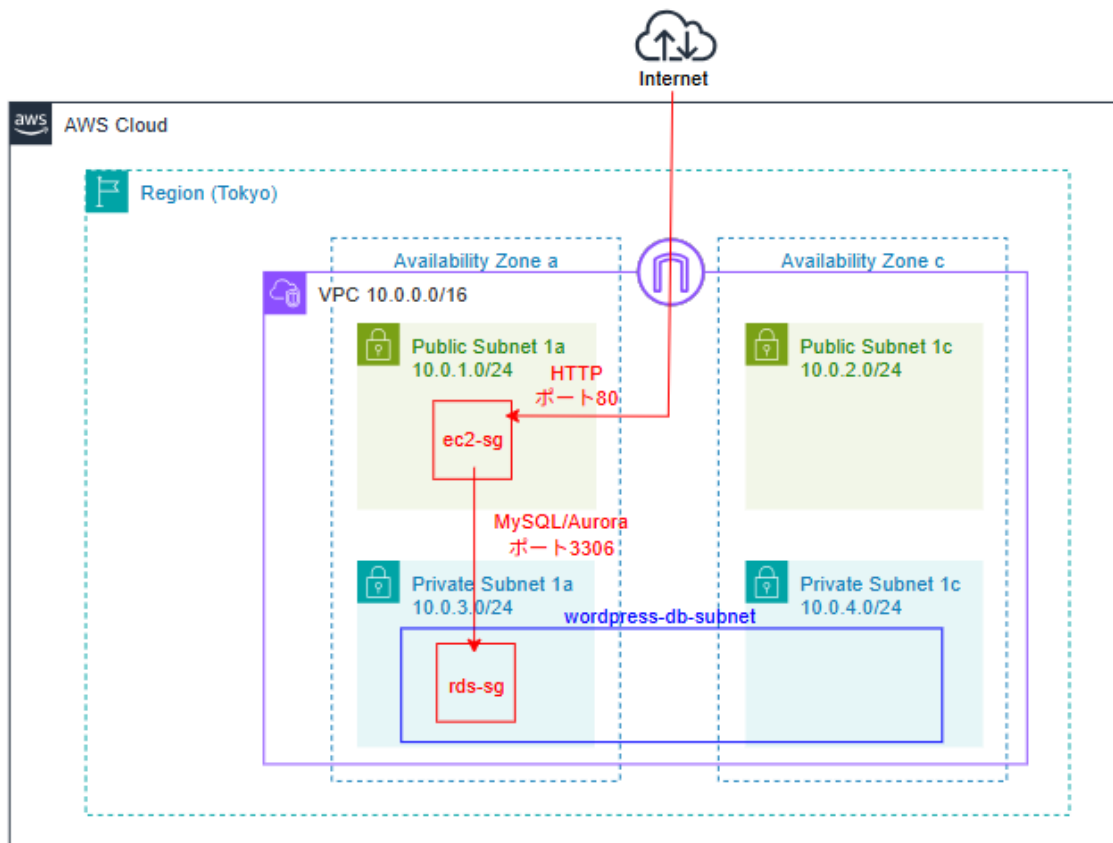
- 手順に記載されているAWSマネジメントコンソールの画面や文言等はコンテンツ作成時点のものとなります。ハンズオンを実施するタイミングによっては差異がある場合がありますのでご注意ください。
- 手順内に記載のない設定項目についてはデフォルトのままにしてください。
- 本資料の第三者への共有はお控えください。

タスクの構築手順

タスク1. VPCリソースの作成

タスクのGOAL

- ✓ 必要なVPCリソースを作成する
- ✓ サービスを起動し、確認を行った後は、必ず都度サービスを停止させる
- ✓ タスク完了後のアーキテクチャは以下の通りです



タスクの流れ

- 1-1. VPCとサブネットを作成する
- 1-2. EC2用のセキュリティグループを作成する
- 1-3. RDS用のセキュリティグループを作成する
- 1-4. RDS用のサブネットグループを作成する

タスクの実施手順

1-1. VPCとサブネットを作成する

1. AWS マネジメントコンソールで画面上部の検索窓から **[VPC]** と入力して表示されたサービス名をクリックします。
2. VPCダッシュボードで、左側にあるメニューから **[VPC]** を選択し **[VPCを作成]** をクリックします。
3. VPCの作成画面が表示されたら、以下のようにVPCの設定を行います。

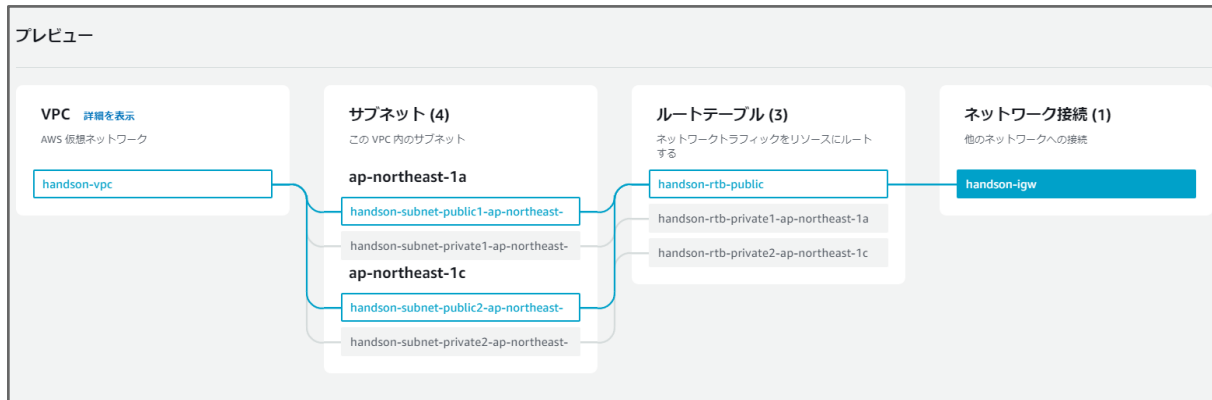
VPCの設定

設定項目	設定値
作成するリソース	VPC など
名前タグの自動生成	「プロジェクト」を「wordpress」に書き換え
IPv4 CIDR ブロック	10.0.0.0/16
アベイラビリティゾーン (AZ) の数	2
パブリックサブネットの数	2
プライベートサブネットの数	2
「サブネットCIDRブロックをカスタマイズ」をクリックして右の通りに指定する	ap-northeast-1aのパブリックサブネット > 10.0.1.0/24 ap-northeast-1cのパブリックサブネット > 10.0.2.0/24 ap-northeast-1aのプライベートサブネット > 10.0.3.0/24 ap-northeast-1cのプライベートサブネット > 10.0.4.0/24
NAT ゲートウェイ	なし
VPC エンドポイント	なし
DNS ホスト名を有効化	チェック

DNS 解決を有効化

チェック

上記の設定後は画面右側にあるプレビューを見て、設定どおりにネットワークが作成されていることを確認しましょう。



4. プレビュー確認後は「VPCを作成」をクリックし、VPCの作成が行われることを確認します。

5. 最後にパブリックサブネット、プライベートサブネットのルーティングが正しく行われていることを確認しましょう。

● VPC

VPCの作成完了後、画面を一度更新してください。

その後、画面左側のメニューからフィルタリングを行うことができます。

「VPCでフィルタリング」から「wordpress-vpc」を選択しましょう。

VPC ダッシュボード

EC2 グローバルビュー

VPC でフィルタリング

検索

- vpc-039508ad7052c587b
- vpc-074f45db66448b082**

インターネットゲートウェイ

Egress-only インターネットゲートウェイ

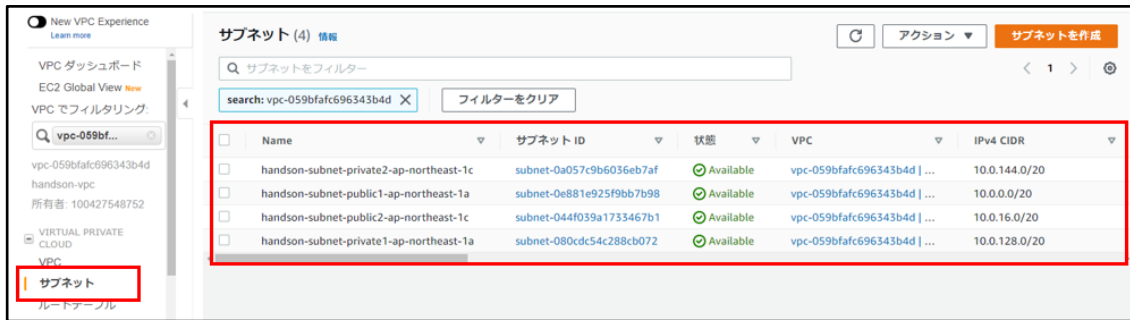
サブネット (7) 情報

属性またはタグでサブネットを検索します

	Name	サブネット ID	状態	VPC
☐	-	subnet-0805d751f32b7a36f	Available	vpc-039508ad7052c587b
☐	-	subnet-03dbd9c68df21cbd2	Available	vpc-039508ad7052c587b
☐	handson-subnet-public1-ap-northeast-1a	subnet-06c6431cc299aa99e	Available	vpc-074f45db66448b082 han...
☐	handson-subnet-private2-ap-northeast...	subnet-05cc405400fda6cee	Available	vpc-074f45db66448b082 han...
☐	-	subnet-001e4520dc40cce7f	Available	vpc-039508ad7052c587b
☐	handson-subnet-public2-ap-northeast-1c	subnet-0a1e3e38fb3f9ad13	Available	vpc-074f45db66448b082 han...
☐	handson-subnet-private1-ap-northeast...	subnet-0b523302902e56161	Available	vpc-074f45db66448b082 han...

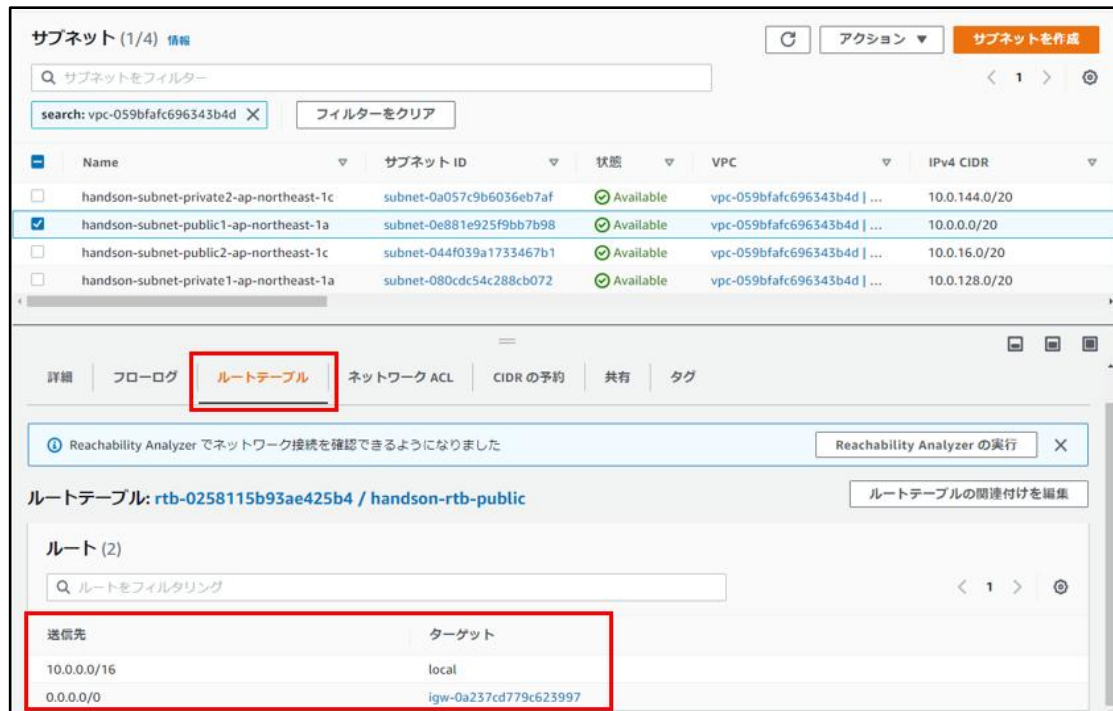
● サブネット

画面左側のメニューから「サブネット」をクリックすると、今回作成したVPC内のサブネットが一覧表示されます。



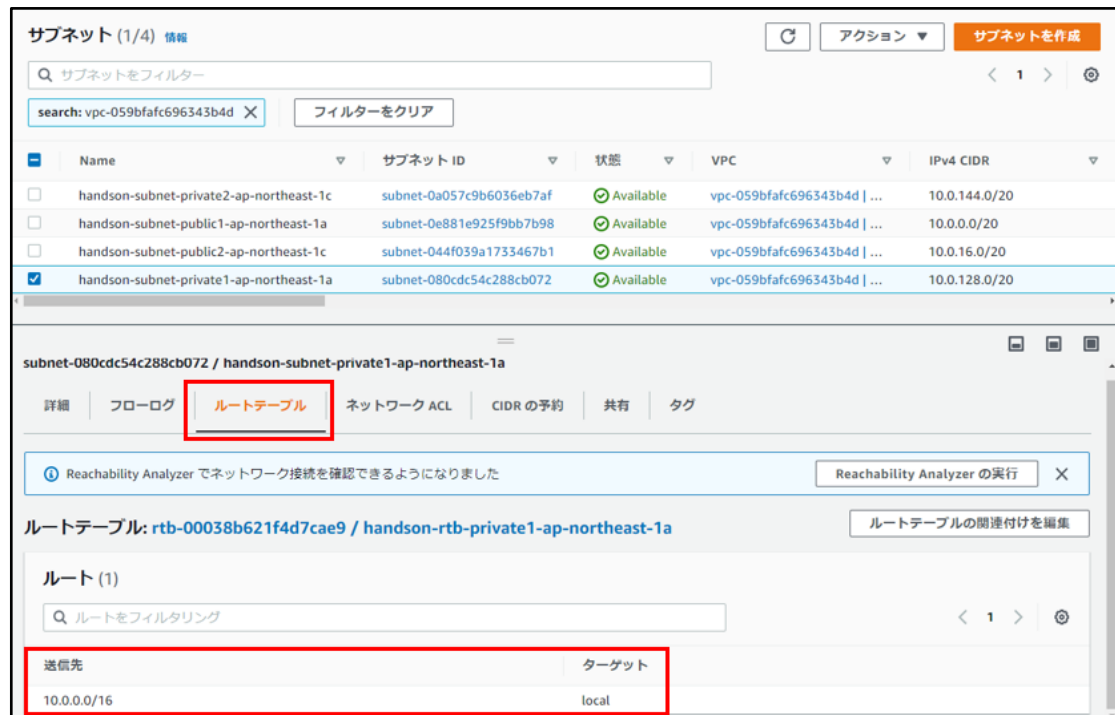
● パブリックサブネットのルートテーブル

サブネットに指定したIPアドレス範囲（10.0.0.0/16）はサブネット内に、それ以外のIPアドレスはインターネットゲートウェイへ伝送していることが分かります。



● プライベートサブネットのルートテーブル

サブネットに指定したIPアドレス範囲（10.0.0.0/16）で経路伝送を行い、インターネットゲートウェイへの経路伝送は行わないことが確認できます。



1-2. EC2用のセキュリティグループを作成する

1. AWS マネジメントコンソールで画面上部の検索窓から「EC2」と入力して表示されたサービス名をクリックします。
2. EC2ダッシュボード左側のメニューから「セキュリティグループ」をクリックします。
3. セキュリティグループの設定画面では、以下の項目を記載の通り設定して「セキュリティグループを作成」をクリックします。

基本的な詳細

設定項目	設定値
セキュリティグループの名前	ec2-sg
説明	ec2-sg
VPC	wordpress-vpc

インバウンドルール

タイプ	プロトコル	ポート番号	ソース
カスタムTCP	TCP	80	Anywhere-IPv4
SSH	TCP	22	3.112.23.0/29

※ 3.112.23.0/29 は、東京リージョンの EC2 Instance Connect 用IPレンジです。EC2 Instance Connect で EC2 に接続するため、このIPレンジからのSSH(22)を許可します。なお、IPレンジは変更の可能性があるため、最新値はAWS公開情報で確認します。

1-3. RDS用のセキュリティグループを作成する

1. セキュリティグループの設定画面で、以下の項目を記載の通り設定して「**セキュリティグループを作成**」をクリックします。

基本的な詳細

設定項目	設定値
セキュリティグループの名前	rds-sg
説明	rds-sg
VPC	wordpress-vpc

インバウンドルール

タイプ	プロトコル	ポート番号	ソース
MYSQL/Aurora	TCP	3306	カスタム > [ec2-sg] を選択

1-4. RDS用のサブネットグループを作成する

1. AWS マネジメントコンソールで画面上部の検索窓から「**RDS**」と入力して表示されたサービス名をクリックします。
2. RDS ダッシュボードの画面左側に表示されている「**サブネットグループ**」をクリックし、「**DBサブネットグループを作成**」をクリックします。

3. サブネットグループの詳細設定では、以下の項目を記載の通り設定して [作成] をクリックします。

サブネットグループの詳細

設定項目	設定値
名前	wordpress-db-subnet
説明	wordpress-db-subnet
VPC	wordpress-vpc

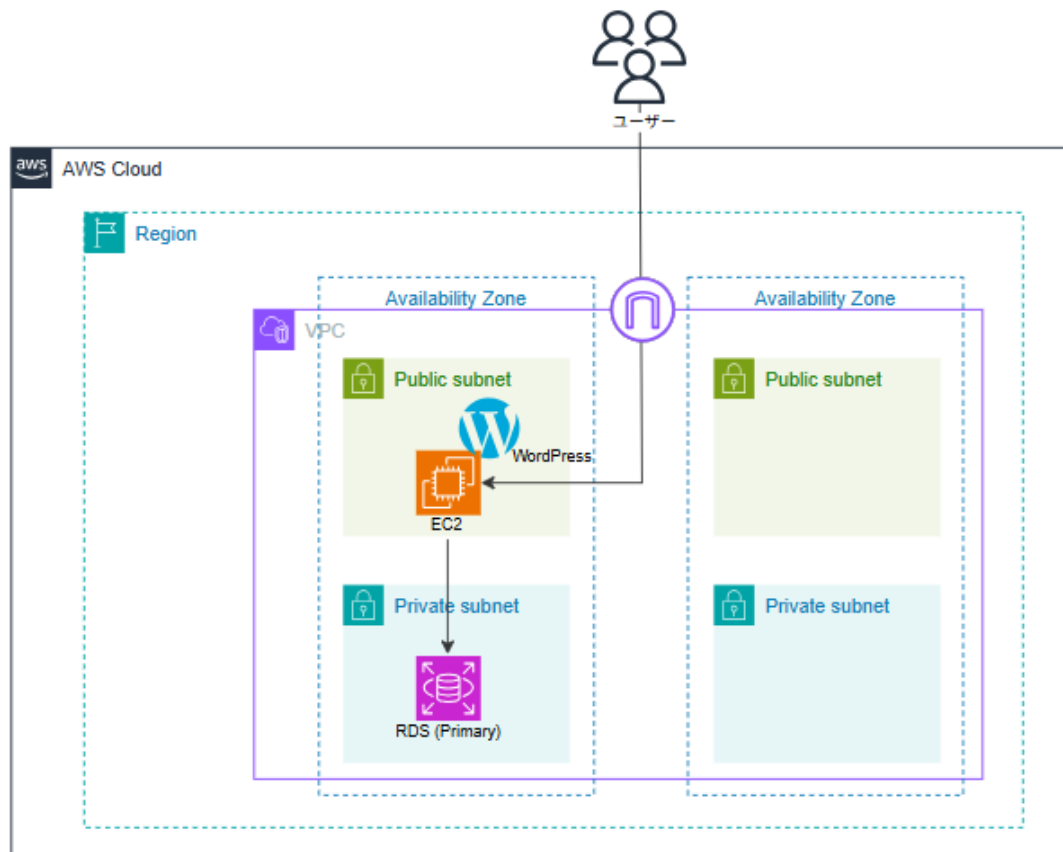
サブネットの追加

設定項目	設定値
アベイラビリティゾーン	以下の2つを選択 • ap-northeast-1a • ap-northeast-1c
サブネット	以下の2つのプライベートサブネットを選択 • CIDRが 10.0.3.0/24 • CIDRが 10.0.4.0/24

タスク2. EC2とRDSの作成

タスクのGOAL

- ✓ RDSインスタンスとEC2インスタンスを作成してWordPressを構築する
- ✓ サービスを起動し、確認を行った後は、必ず都度サービスを停止させる
- ✓ タスク完了後のアーキテクチャは以下の通りです



タスクの流れ

- 2-1. RDSインスタンスを作成する
- 2-2. EC2インスタンスを作成する
- 2-3. WordPressを構築する

タスクの実施手順

2-1. RDSインスタンスを作成する

1. AWS マネジメントコンソールで画面上部の検索窓から **[RDS]** と入力して表示されたサービス名をクリックします。
2. RDS ダッシュボードに表示されている **[データベースを作成]** をクリックします。
3. インスタンスの詳細設定では、以下の項目を記載の通り設定して **[データベースの作成]** をクリックします。

データベース作成方法を選択

設定項目	設定値
データベース作成方法	標準作成

エンジンのオプション

設定項目	設定値
エンジンのタイプ	MySQL
バージョン	MySQL 8.0.41

テンプレート

設定項目	設定値
テンプレート	無料利用枠

可用性と耐久

設定項目	設定値
デプロイオプション	シングルAZ DBインスタンスデプロイ (1インスタンス)

設定

設定項目	設定値
DB インスタンス識別子	wordpress-rds
マスターユーザー名	admin
マスターパスワード	systemsss
パスワードを確認	systemsss

インスタンスの設定

設定項目	設定値
DB インスタンスクラス	バースト可能クラス (t クラスを含む) > db.t3.micro

ストレージ

設定項目	設定値
ストレージタイプ	汎用SSD (gp3)
ストレージ割り当て	20

接続

設定項目	設定値
コンピューティングリソース	EC2コンピューティングリソースに接続しない
ネットワークタイプ	IPv4
Virtual Private Cloud (VPC)	wordpress-vpc
DBサブネットグループ	wordpress-db-subnet
パブリックアクセス	なし
VPC セキュリティグループ	[x]を押して default を削除 > rds-sg を選択
アベイラビリティーゾーン	ap-northeast-1a

データベース認証

設定項目	設定値
データベース認証オプション	パスワード認証

追加設定

設定項目	設定値
最初のデータベース名	wordpress
バックアップ	チェックなし

※ マスターユーザー名、マスターパスワード、最初のデータベース名の記載を誤ると後続の手順でデータベースに接続することができないため注意してください

※ 「データベースの作成」をクリック後に表示される「推奨アドオン」は「閉じる」をクリックして閉じてください

データベースの作成が開始されると 5～10 分で利用開始することができます。ここでは利用開始になるのを待たずに次のステップへ進みましょう。

2-2. EC2インスタンスを作成する

1. AWS マネジメントコンソールで画面上部の検索窓から「EC2」と入力して表示されたサービス名をクリックします。
2. EC2ダッシュボード左側のメニューから「インスタンス」をクリックします。
3. 画面右上の「インスタンスを起動」をクリックします。
4. インスタンスの起動画面では、以下の内容に従って設定を行きましょう。

名前とタグ

設定項目	設定値
名前	WordPress 01

アプリケーションおよびOSイメージ (Amazonマシンイメージ)

設定項目	設定値
Amazon マシンイメージ (AMI)	Amazon Linux 2023 (kernel-6.12)
アーキテクチャ	64ビット (x86)

インスタンスタイプ

設定項目	設定値
インスタンスタイプ	t3.micro

ネットワーク設定

設定欄の右上にある **編集** ボタンをクリックして以下の通りに設定。

設定項目	設定値
VPC	wordpress-vpc
サブネット	wordpress-subnet-public1-ap-northeast-1a
パブリックIPの自動割り当て	有効化
ファイアウォール (セキュリティグループ)	既存のセキュリティグループを選択する
共通のセキュリティグループ	ec2-sg

ストレージ設定

設定項目	設定値
ルートボリューム	8 GiB gp2

5. これまでに行った設定内容を確認して **インスタンスを起動** をクリックします。

- インスタンスの状態が「**実行中**」になれば、このステップは完了です。

1. EC2のインスタンス一覧から WordPress01 を選択して 画面上部の **[接続]** をクリックします。

- ```
#_
#####
~\ #####\
~\ #####|
~\ \###/
~\ \#/
~\ v-'\-->
~\ /
~\ /m/'
Last login: Tue Aug 19 06:49:58 2025 from 3.112.23.4
[ec2-user@ip-172-31-5-184 ~]$
```
- Amazon Linux 2023
- <https://aws.amazon.com/linux/amazon-linux-2023>

- ```
# システムパッケージをアップデート
sudo dnf update -y
```

```
# WordPress 実行に必要なになる PHP とライブラリをインストール
sudo dnf install php php-cli php-common php-mbstring php-xml php-gd php-mysqlnd php-curl php-zip -y

# MySQLレポジトリとGPGキー、クライアントのインストール
sudo dnf install https://dev.mysql.com/get/mysql80-community-release-el9-5.noarch.rpm -y
sudo rpm --import https://repo.mysql.com/RPM-GPG-KEY-mysql-2022
sudo dnf install mysql-community-client -y

# 初期インストールされているApacheの起動・自動起動設定
sudo systemctl enable httpd.service
sudo systemctl start httpd.service

# WordPress をダウンロードして解凍する
wget https://ja.wordpress.org/latest-ja.tar.gz
tar -xzf latest-ja.tar.gz

# Web サーバー上で公開するためのファイルの複製と権限設定
sudo cp -r ~/wordpress/* /var/www/html/
sudo chown apache:apache -R /var/www/html
```

4. EC2 インスタンスのパブリックIPアドレスへアクセスすると WordPress のセットアップ画面が表示されるので「さあ、始めましょう！」をクリックします。
5. データベースへの接続情報の入力を求められたら、以下のように記載を行い「送信」をクリックします。

項目	値
データベース名	wordpress

ユーザー名	admin
パスワード	systemsss
データベースのホスト名	RDS のエンドポイントを指定 例: sample.abcdef12gh56.ap-northeast-1.rds.amazonaws.com
テーブル接頭辞	wp_

- 遷移後の画面でデータベースとの接続が確認出来たら **［インストール実行］** をクリックします。
- 表示された画面で以下を参考に必要情報を入力し **［WordPressをインストール］** をクリックします。

項目	値
サイトのタイトル	任意のタイトル
ユーザー名	admin
パスワード	初期生成されているものを利用
メールアドレス	ご自身のメールアドレス
検索エンジンでの表示	チェックを入れる（検索されないようにする）

※ パスワードは後で利用するため必ず控えておいてください。

- 設定が成功したら **［ログイン］** をクリックし、手順7で指定/確認した ユーザー名 とパスワードでログインを行います。

動作確認

最後に以下の手順に従って WordPress を利用してサンプルのブログ記事を投稿してみましょう。

- WordPressのインストールが完了したら「ログイン」をクリックし、ユーザー名とパスワードでログインを行う
- WordPress の管理画面の左にあるメニューから **［設定］** > **［パーマリンク］** とクリックする

- パーマリンクの設定画面が表示されたら共通設定の一覧から「基本」を選択して「変更を保存」をクリックする
- メニューの「投稿」から「投稿を追加」をクリックする
- 投稿の編集画面が表示されたら任意のタイトルを入力する
- 続けて任意の本文を入力して改行を行い「+」ボタンを押して任意の画像を挿入する
- 画面右上にある「公開」を2回続けてクリックする
- 「投稿を表示」を押して作成した記事を確認する
 - 記事内の画像を右クリックして別のタブで開くと画像の URL を確認できる
 - EC2 インスタンスで以下のコマンドを実行すると画像ファイルを確認することができます

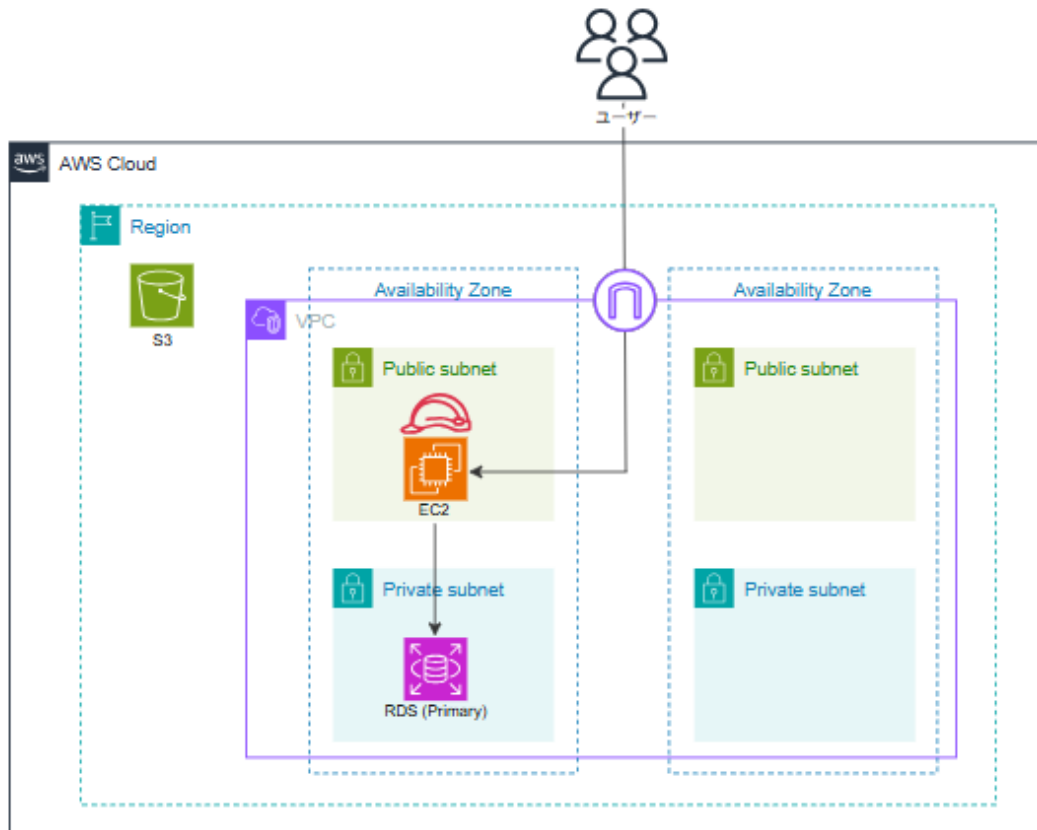
```
ls -la /var/www/html/wp-content/uploads/yyyy/mm/
```

※ yyyy、mm は作業日に合わせて年と月を指定してください

タスク3. S3の作成とIAMロールの利用

タスクのGOAL

- ✓ S3バケットを作成してEC2インスタンスから利用可能にする
- ✓ サービスを起動し、確認を行った後は、必ず都度サービスを停止させる
- ✓ タスク完了後のアーキテクチャは以下の通りです



タスクの流れ

- 3-1. S3バケットを作成する
- 3-2. IAMロールを作成しEC2インスタンスへ割り当てる
- 3-3. S3を利用するためのプラグインをWordPressへ追加する

タスクの実施手順

3-1. S3バケットを作成する

1. AWS マネジメントコンソールで **[サービス]** から **[S3]** をクリックします。
2. S3 コンソールへ遷移したら **[バケットを作成]** をクリックします。
3. **[一般的な設定]** のセクションで、バケット名、AWS リージョンを次のように入力します。

設定項目	設定値
バケット名	<i>name-YYYYMMDD</i> ※ tanaka-20220401など
AWSリージョン	アジアパシフィック（東京）

※バケット名は世界中で被らない一意の名称を指定する必要があります。
作成時にエラーとなった場合は数字や文字列を末尾に付け加えるなどしてください。

4. **[オブジェクト所有者]** のセクションで **[ACL有効]** を選択します。
5. **[このバケットのブロックパブリックアクセス設定]** のセクションで、以下の設定を行ってください。
 - パブリックアクセスをすべてブロック： **チェックを外す**
 - 現在の設定により、このバケットとバケット内のオブジェクトが公開される可能性があることを承認します。： **チェックを付ける**
6. ページ下部の **[バケットを作成]** をクリックします。
7. バケットが正常に作成されたことを確認します。

3-2. IAMロールを作成しEC2インスタンスへ割り当てる

1. AWS マネジメントコンソールで **[サービス]** から **[IAM]** をクリックします。

2. IAMコンソールから **[ロール]** をクリックし、**[ロールを作成]** をクリックします。

3. 下記の設定を行い、**[次のステップ: アクセス権限]** ボタンを押下します。

設定項目	設定値
信頼されたエンティティタイプ	AWSのサービス
一般的なユースケース	EC2

4. ロールにアタッチするポリシーとして、検索窓から「s3full」と検索して表示される **[AmazonS3FullAccess]** にチェックを付け **[次へ]** をクリックします。

5. ロールの詳細欄で以下のロール名を入力し、**[ロールを作成]** ボタンをクリックします。

設定項目	設定値
ロール名	ec2-wordpress-role

6. AWS マネジメントコンソールで **[サービス]** から **[EC2]** をクリックします。

7. 画面左のナビゲーションペインから **[インスタンス]** をクリックし、**[WordPress 01]** を選択します。

8. **[アクション]** から **[セキュリティ]** を選び、**[IAM ロールを変更]** をクリックします。

9. IAM ロールから **[ec2-wordpress-role]** を選択し、**[IAMロールの更新]** をクリックします。

3-3. S3を利用するためのプラグインをWordPressへ追加する

1. 以下の URL を参考にWeb ブラウザから WordPress 01 インスタンスの WordPress 管理者ページへアクセスします。

➤ <http://ec2のパブリックIPアドレス/wp-login.php>

2. ユーザー名とパスワードを指定してログインします。

設定項目	設定値
ユーザー名	admin
パスワード	WordPressの作成時に控えたパスワード

- 管理画面の左側にあるメニューから **［プラグイン］** > **［プラグインを追加］** とクリックします。
- 検索窓に **［WP Offload Media Lite for Amazon S3］** と入力して検索を行い、表示された **［Amazon S3、DigitalOcean Spaces、Google Cloud Storage 用の WP Offload Media Lite］** の欄にある **［今すぐインストール］** をクリックします。
- インストールが完了するとインストールボタンが「有効化」に変わるので、**［有効化］** をクリックします。
- 遷移したプラグインの一覧画面で、WP Offload Media Lite の **［Settings］** をクリックします。
- 「1.Select Provider」欄で「Amazon S3」のラジオボタンを選択します。
- 「2.Connection Method」欄で**［My server is on Amazon Web Services and I'd like to use IAM Roles］** のラジオボタンを選択して、**［Save&Continue］** をクリックします。
- 「1.New or Existing Bucket?」欄で「Use Existing Bucket」を選びます。
- 「2.Select Bucket」欄で**［Browse existing buckets］** を選択します。
- 利用可能な S3 バケットの一覧が表示されるので、Step 1 で作成した S3 バケットを選択して **［Save Selected Bucket］** をクリックします。
- 遷移後の画面はデフォルトのまま「Keep Bucket Security As Is」をクリックします。

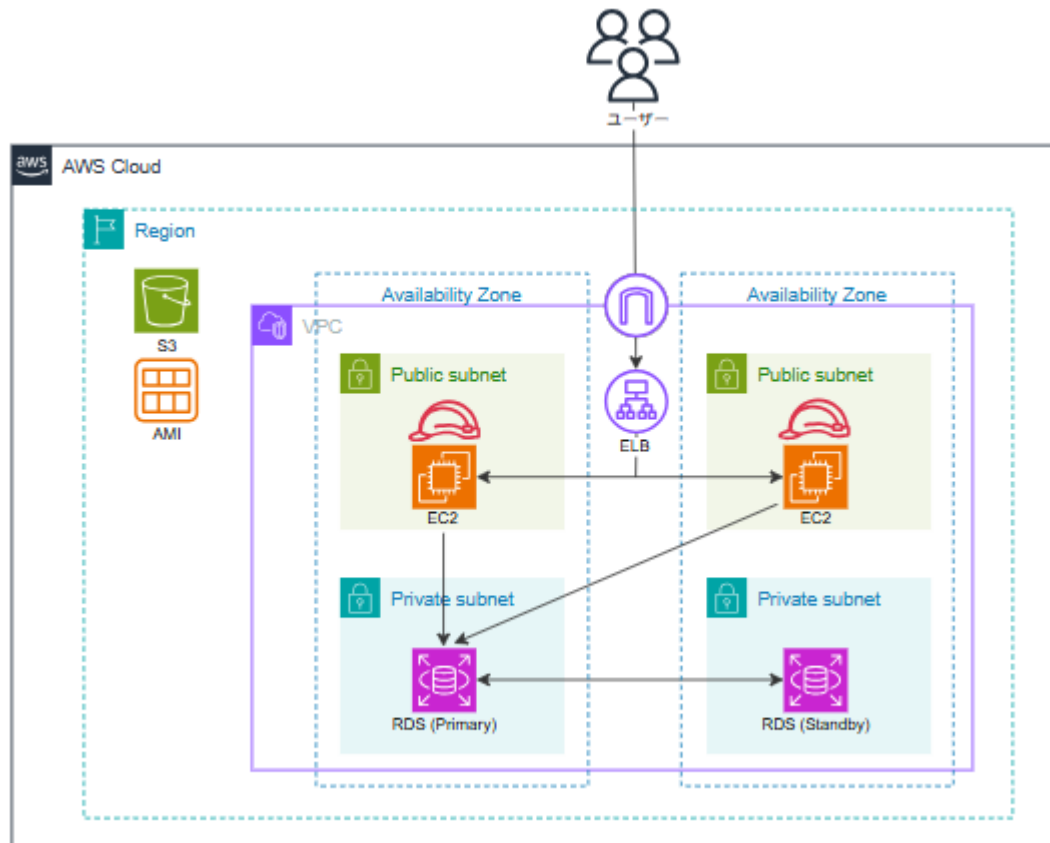
動作確認

- WordPress の管理画面で、画面左のメニューから [投稿] > [新規追加] をクリックする
- 記事のタイトルと本文を入力し、[+] ボタンをクリックして画像を挿入する
※このとき挿入する画像は新規で新しくアップロードしてください
- 記事の作成が終わったら、画面右上にある [公開] を2回続けてクリックする
- [投稿を表示] をクリックして投稿した記事を表示する
- 記事内の画像を右クリックして別タブで開くと、画像の URL が S3 バケットの URL であることを確認できます

タスク4. ELBの作成と環境の冗長化

タスクのGOAL

- ✓ ELBやAMIを利用してマルチAZ環境を構築する
- ✓ サービスを起動し、確認を行った後は、必ず都度サービスを停止させる
- ✓ タスク完了後のアーキテクチャは以下の通りです



タスクの流れ

- 4-1. ALBを作成する
- 4-2. カスタムAMIを作成してEC2を複製する
- 4-3. RDSをマルチAZ化する

タスクの実施手順

4-1. ALBを作成する

1. AWS マネジメントコンソールで **［サービス］** から **［EC2］** をクリックします。
2. 左側のナビゲーションペインで **［ロードバランサー］** をクリックします。
3. **［ロードバランサーの作成］** をクリックします。
4. **［ロードバランサータイプ］** で **［Application Load Balancer］** の **［作成］** をクリック します。
5. Application Load Balancer の作成画面が表示されたら、以下の内容を指定します。

基本的な設定

設定項目	設定値
ロードバランサー名	wordpress-elb
スキーム	インターネット向け
IPアドレスタイプ	IPv4

ネットワークマッピング

設定項目	設定値
VPC	wordpress-vpc
マッピング	以下の2つを選択 ・ ap-northeast-1a > パブリックサブネット-1a ・ ap-northeast-1c > パブリックサブネット-1c

6. セキュリティグループの設定では **［新しいセキュリティグループを作成］** をクリックし、別タブで開いたセキュリティグループの作成画面では以下の指定を行い **［セキュリティグループの作成］** をクリックします。

基本的な詳細

設定項目	設定値
名前	elb-sg
説明	elb-sg
VPC	wordpress-vpc

インバウンドルール

タイプ	プロトコル	ポート番号	ソース
HTTP	TCP	80	Anywhere-IPv4

7. セキュリティグループの作成が完了したら、ロードバランサーの設定を行っているタブに戻り、更新ボタンをクリックします。するとプルダウンに作成した **[elb-sg]** が表示されるため **[elb-sg]** をクリックしてください。また、**[default]** は **[×]** を押して削除しておきます。
8. リスナーとルーティングの設定では **[ターゲットグループの作成]** をクリックし、別タブで開いたターゲットグループの作成画面では「**基本的な設定**」と「**ヘルスチェック**」欄で以下のように指定を行い「**次へ**」をクリックします。

基本的な設定

設定項目	設定値
ターゲットタイプの選択	インスタンス
ターゲットグループ名	wordpress-tg
プロトコル	HTTP
ポート	80
VPC	wordpress-vpc
プロトコル	HTTP1

ヘルスチェック

設定項目	設定値
ヘルスチェックプロトコル	HTTP
ヘルスチェックパス	/login.php
ヘルスチェックの詳細設定 ＞ 正常のしきい値	2
ヘルスチェックの詳細設定 ＞ 間隔	30

9. ターゲットの登録画面では「使用可能なインスタンス」の欄に表示されている EC2 インスタンス [WordPress 01] にチェックを入れ、ポートに [80] を指定して [保留中として以下を含める] をクリックします。
10. ターゲットに WordPress 01 が追加されたら [ターゲットグループの作成] をクリックします。
11. ターゲットグループの作成が完了したらロードバランサーの設定を行っているタブに戻り、更新ボタンをクリックします。するとプルダウンに作成した [handson-tg] が表示されるため [handson-tg] をクリックしてください。
12. 以上で、ELB を作成するための設定はすべてになるため、最後に [ロードバランサーの作成] をクリックします。
13. 「ステータス」が「active」になるまで3～4分程度待つて画面右上隅にある更新アイコンをクリックします。
13. ロードバランサーが active になったら画面左側のナビゲーションペインから [ターゲットグループ] をクリックします。
14. ターゲットグループの一覧から handson-tg をクリックし、画面下部の「ターゲット」タブ内の「登録済みターゲット」に表示されているインスタンスのヘルスステータスが [healthy] になるまで定期的に更新ボタンをクリックして待ちます。

- 15.ヘルスステータスが [healthy] になった確認ができれば、再び画面左のナビゲーションペインから [ロードバランサー] をクリックします。
16. ロードバランサーの一覧から handson-elb を選択し、画面下部の「詳細」タブに表示されている [DNS名] の値をコピーします。
17. ブラウザの新しいタブにコピーしたDNS名を貼り付けて Enter キーを押し、WordPress の画面が表示されることを確認します。
- 18.最後に ELB 経由で WordPress へアクセスした場合でも、正常にリクエストを返せるようにサイトのアドレス設定を変更します。
- 19.WordPress の管理者用ページで画面左のメニューから [設定] をクリックします。
- 20.「WordPress アドレス (URL)」と「サイトアドレス (URL)」を ELB の DNS 名へ変更します。
 - `http://<ロードバランサーの DNS 名>`

動作確認

- ALBの構築が完了したらロードバランサーのDNS名にHTTPアクセスを行い、WordPressの画面が表示されることを確認する
- 最後に ELB 経由で WordPress へアクセスした場合でも、正常にリクエストを処理できるように以下の手順でサイトのアドレス設定を変更します
 - WordPress の管理者用ページを表示して画面左のメニューから [設定] をクリックする
 - 「WordPress アドレス (URL)」と「サイトアドレス (URL)」を ELB の DNS 名へ変更する
`http://<ロードバランサーのDNS名>`

4-2. カスタムAMIを作成してEC2を複製する

1. EC2 ダッシュボードの左側にあるナビゲーションペインから **［インスタンス］** をクリックします。
2. インスタンスの一覧から「Wordpress 01」を選択して **［アクション］** > **［イメージとテンプレート］** > **［イメージを作成］** の順にクリックします。
3. イメージの作成画面では、以下の項目を記載の通りに設定して **［イメージを作成］** をクリックします。

設定項目	設定値
イメージ名	WordPress AMI
イメージの説明	WordPress AMI

4. AMI の作成を行ったら、画面左側のナビゲーションペインから **［AMI］** を選択します。
5. AMI の一覧から作成した AMI のステータスが「利用可能」になるまで3分ほど待ちます。
6. AMI のステータスが「利用可能」になったら、画面左側のナビゲーションペインから **［インスタンス］** を選択し、**［インスタンスを起動］** をクリックします。
7. 以下の設定を行ってインスタンスを作成してください。
指定していない項目はデフォルトのままで問題ありません。

設定項目	設定値
名前	WordPress 02
アプリケーションおよびOS イメージ	自分のAMI > 自己所有 > WordPress AMI
インスタンスタイプ	t2.micro
キーペア	キーペアなしで続行

ネットワーク設定 ＞ VPC	wordpress-vpc
ネットワーク設定 ＞ サブネット	パブリックサブネット-1c
自動割り当てパブリック IP	有効化
セキュリティグループ	既存のセキュリティグループを選択する ＞ ec2-sg

8. EC2 インスタンスの作成が完了し、ステータスチェックが「2/2 のチェックに合格しました」と表示されたら、EC2 インスタンスのパブリック IP アドレスを利用して WordPress サイトが表示されることを確認します。

続いて今作成したEC2 インスタンスを ELB のターゲットに登録し、2 台の EC2 インスタンスへの処理の分散を行います。

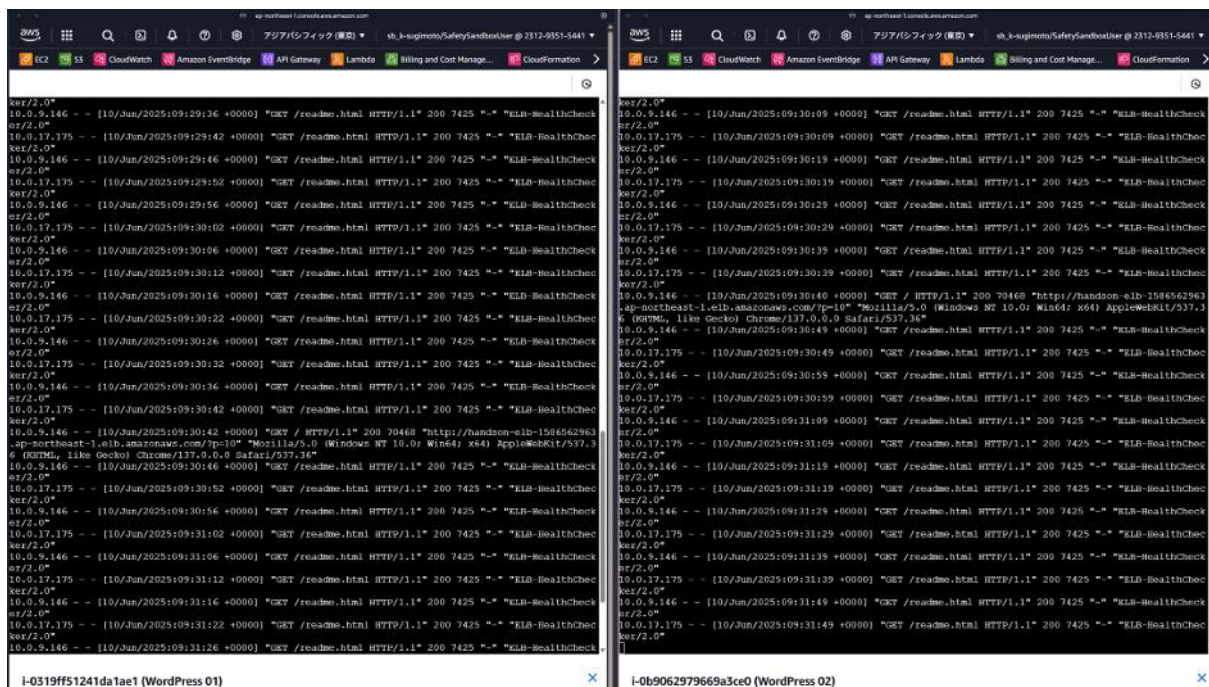
1. EC2 ダッシュボードの左側にあるナビゲーションペインから [ターゲットグループ] をクリックします。
2. ターゲットグループの一覧から Step 1 で作成した「handson-tg」を選択して「ターゲット」タブ内の [ターゲットを登録] をクリックします。
3. ターゲットの登録画面では「使用可能なインスタンス」の欄に表示されている EC2 インスタンス [WordPress 02] にチェックを入れ、ポートに [80] を指定して [保留中として以下を含める] をクリックします。
4. ターゲットに WordPress 02 が追加されたら [保留中のターゲットの登録] をクリックします。
5. ターゲットグループの作成が完了したら登録済みのターゲット一覧を見て、登録した EC2 インスタンスのステータスが「healthy」になるまで待ちます。
6. EC2 インスタンスのステータスが「healthy」になったのを確認できたら、ELB の DNS 名を利用してアプリケーションへアクセスを行い、アプリケーションが正しく表示されることを確認します。

動作確認

- ELBによってリクエストの振り分けが行われていることを確認するために、2つのタブを開いてEC2 Instance Connect を利用して2つの EC2 インスタンスへそれぞれアクセスします。
- アクセスできたら以下のコマンドを実行しましょう

```
sudo tail -f /var/log/httpd/access_log
```

- その状態で ELB の DNS 名に複数回アクセスすると、アクセスログがそれぞれの EC2 インスタンスで出力されていることを確認できます



4-3. RDSをマルチAZ化する

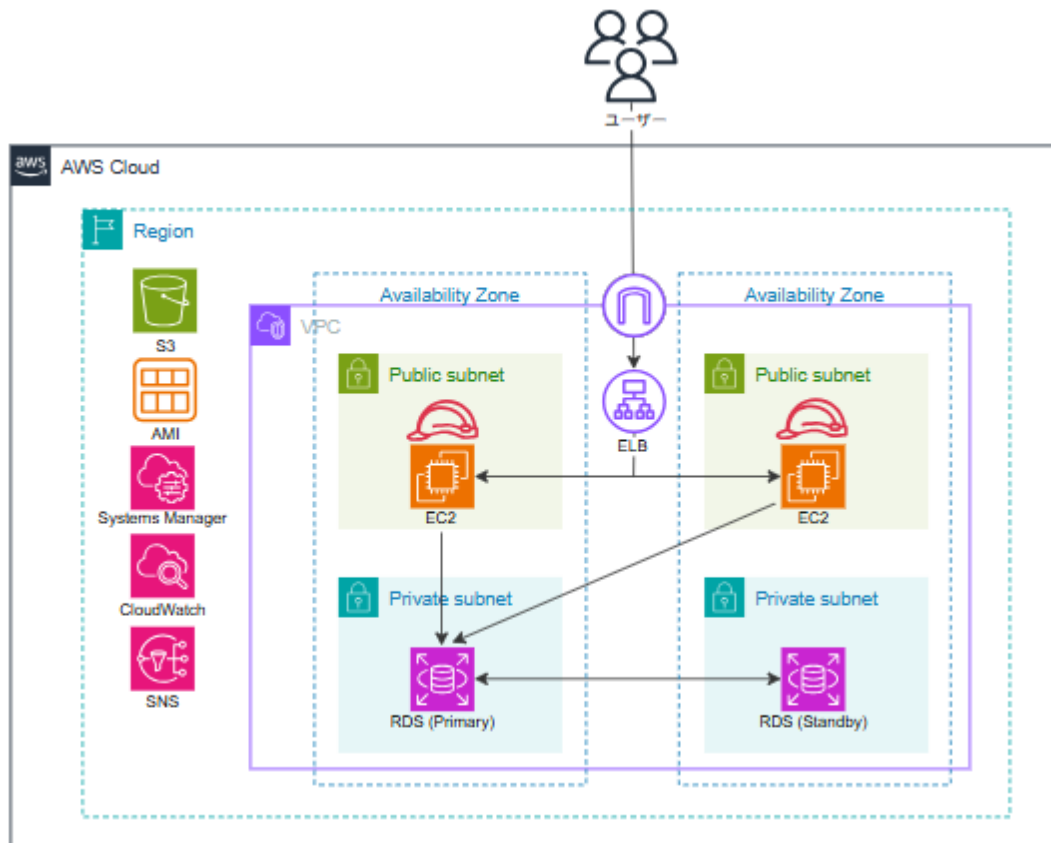
1. AWS マネジメントコンソールで画面上部の検索窓から [RDS] と入力して表示されたサービス名をクリックします。
2. RDSダッシュボード左側のメニューから [データベース] をクリックします
3. データベースの一覧から作成されている RDS インスタンス (handson-rds) を選択して [変更] をクリックします。

4. 可用性と耐久セクションの「マルチ AZ 配置」の欄にある **「スタンバイインスタンスを作成する（本稼働環境向けに推奨）」** を選択し、画面下部の **「続行」** をクリックします。
5. 変更のスケジューリングで **「すぐに適用」** をチェックして **「DB インスタンスを変更」** をクリックします。（警告メッセージが表示された場合、今回の演習では無視頂いて大丈夫です）
6. RDS インスタンスのステータスが「変更中」になります。5 分 ～ 15 分程度でステータスが「利用可能」になると マルチ AZ 化が完了です。
マルチ AZ 化の進捗状況は RDS インスタンス（handson-rds）をクリックして「ログとイベント」タブ 内の最近のイベントから参照できます。

タスク5. SystemsManagerとCloudWatchの利用

タスクのGOAL

- ✓ CloudWatchの利用設定を行ってカスタムメトリクスやログを監視する
- ✓ サービスを起動し、確認を行った後は、必ず都度サービスを停止させる
- ✓ タスク完了後のアーキテクチャは以下の通りです



タスクの流れ

- 5-1. IAMロールへIAMポリシーを追加する
- 5-2. SystemsManagerを利用したCloudWatchエージェントの設定と起動
- 5-3. CloudWatchのメトリクスを利用したアラートを設定する
- 5-4. CloudWatch Logsを確認する

タスクの実施手順

5-1. SystemsManagerを利用したCloudWatchエージェントの設定と起動

1. AWS マネジメントコンソールで [サービス] から [IAM] をクリックします。
2. IAMコンソールから [ロール] をクリックし、検索欄に [ec2-wordpress-role] を指定して検索を行い、表示されたロール名をクリックします。
3. 許可のタブ内にある「許可を追加」から「ポリシーをアタッチ」をクリックします。
4. 以下のIAMポリシーを検索してチェックをつけて、画面下部の「許可を追加」をクリックします
 - CloudWatchAgentAdminPolicy
 - AmazonSSMManagedInstanceCore

5-2. SystemsManagerを利用したCloudWatchエージェントの設定と起動

1. AWS マネジメントコンソールで [サービス] から [Systems Manager] をクリックします。
2. 画面左側のメニューから [Run Command] をクリックします。
3. オレンジ色の「Run Command」ボタンをクリックします。
4. コマンドドキュメントの検索欄に「AWS-ConfigureAWSPackage」と入力して検索を行います。
5. 表示された「AWS-ConfigureAWSPackage」のラジオボタンを選択して、ドキュメントバージョンは「ランタイムの最新バージョン」を選択します。
6. 各設定欄に以下の値を入力して「実行」をクリックします。

コマンドのパラメータ

設定項目	設定値
Action	Install
Installation Type	Uninstall and reinstall
Name	AmazonCloudWatchAgent

ターゲット

設定項目	設定値
ターゲット	インスタンスを手動で選択する
インスタンス	以下のインスタンスを選択する ・ WordPress 01 ・ WordPress 02

出力オプション

設定項目	設定値
S3 バケットへの書き込みを有効化する	オフ（チェックを外す）

7. 10 秒ほど経過したらリロードボタンを押して、Run Command の実行状況を確認して下さい。

続いて複数の EC2 インスタンスから CloudWatch Agent の設定ファイルを共同で利用できるように、CloudWatch Agent の設定ファイルを Systems Manager の Parameter Store にアップロードします。

1. 画面左側のメニューから [パラメータストア] をクリックします。
2. 「パラメータの作成」 をクリックします。

3. 以下の値を入力して「パラメータを作成」 をクリックします。

設定項目	設定値
名前	AmazonCloudWatch-Agentconf
説明	AmazonCloudWatch-Agentconf
利用枠	標準
タイプ	文字列
データ型	text
値	AmazonCloudWatch-Agentconf. jsonの内容を転記する

CloudWatch Agent を実行して、カスタムメトリクスとログの送信を開始するにあたって、collectdの利用準備を行うコマンドをEC2インスタンスに対して実行します。

1. 画面左側のメニューから [Run Command] をクリックします。
2. オレンジ色の「Run Command」 ボタンをクリックします。
3. コマンドドキュメントの検索欄に「AWS-RunShellScript」と入力して検索を行います。
4. 表示された「AWS-RunShellScript」 のラジオボタンを選択して、ドキュメントバージョンは「ランタイムの最新バージョン」を選択します。
5. 各設定欄に以下の値を入力して「実行」 をクリックします。

コマンドのパラメータ

```
sudo mkdir -p /usr/share/collectd/; sudo touch /usr/share/collectd/types.db
```

ターゲット

設定項目	設定値
ターゲット	インスタンスを手動で選択する
インスタンス	以下のインスタンスを選択する ・ WordPress 01 ・ WordPress 02

出力オプション

設定項目	設定値
S3 バケットへの書き込みを有効化する	オフ（チェックを外す）

6. 数秒経過したらリロードボタンを押して、Run Command の実行状況を確認して下さい。

最後に CloudWatch Agent を実行し、カスタムメトリクスとログの送信を開始します。

1. 画面左側のメニューから [Run Command] をクリックします。
2. オレンジ色の「Run Command」ボタンをクリックします。
3. コマンドドキュメントの検索欄に「AmazonCloudWatch-ManagedAgent」と入力して検索を行います。
4. 表示された「AmazonCloudWatch-ManagedAgent」のラジオボタンを選択して、ドキュメントバージョンは「ランタイムの最新バージョン」を選択します。
5. 各設定欄に以下の値を入力して「実行」をクリックします。

コマンドのパラメータ

設定項目	設定値
Action	configure
Mode	ec2
Optional Configuration Source	ssm
Optional Configuration Location	AmazonCloudWatch-Agentconf
Optional Restart	yes

ターゲット

設定項目	設定値
ターゲット	インスタンスを手動で選択する
インスタンス	以下のインスタンスを選択する • WordPress 01 • WordPress 02

出力オプション

設定項目	設定値
S3 バケットへの書き込みを有効化する	オフ（チェックを外す）

6. 数秒経過したらリロードボタンを押して、Run Command の実行状況を確認して下さい。

動作確認

- コマンドの実行後に数分待ってからCloudWatchダッシュボードにアクセスする
- 画面左側のメニューの「すべてのメトリクス」から「CWAgent」を選択し、設定ファイルで指定したカスタムメトリクス項目が取得されていることを確認する

5-3. CloudWatchのメトリクスを利用したアラートを設定する

1. CloudWatch マネジメントコンソールで、ナビゲーションペインから **【すべてのアラーム】** をクリックし **【アラームの作成】** をクリックします。
2. アラームの作成画面が開いたら **【メトリクスの選択】** をクリックします。
3. メトリクスの選択ダイアログが開いたら **【EC2】** > **【インスタンス別メトリクス】** と選択し、検索窓にインスタンス ID を入力して Enter キーを押します。
4. インスタンス ID と一致するインスタンスの **【CPUUtilization】** をチェックし、**【メトリクスの選択】** をクリックします。
5. メトリクスと条件の指定画面が表示されたら条件の欄に以下の値を指定して **【次へ】** をクリックします。（CPU使用率が 50% を超えた場合にアラームを発報する設定です）

条件

設定項目	設定値
しきい値の種類	静的
CPU Utilizaions が次の時…	より大きい
…よりも	50

6. アクションの設定画面の通知欄に以下の値を指定して **【新しいトピックの作成】** をクリックします。

設定項目	設定値
アラーム状態トリガー	アラーム状態
SNS トピックの選択	新しいトピックの作成
新規トピックの作成中	monitoring-alarm

通知を受け取る E メールエンドポイント	自分のメールアドレス
----------------------	------------

7. トピックの作成が完了したら **「次へ」** をクリックします。
8. **「名前と説明」** で以下の値を指定して **「次へ」** をクリックします。

設定項目	設定値
アラーム名	Monitoring CPU Utilization

9. プレビュー画面で設定を確認して **「アラームの作成」** をクリックします。
10. 通知先のメールアドレスとして指定したメールの受信ボックスを確認します。
11. 受信した確認メールを開き、[Confirm subscription]というリンクをクリックして開きます。 Webページが開き「Subscription confirmed!」というメッセージが表示されることを確認します。
12. CloudWatch コンソールで、アラームのステータスが **「OK」** になっていることを確認します。（メトリクスの収集数が不足している場合は **「データ不足」** と表示されますが数分待つと **「OK」** と表示されます）
13. 最後に EC2 インスタンスに接続して以下のコマンドを実行します。コマンド実行後に数分待つことでアラームの発砲を確認することができます。

```
# 簡易負荷ツール stress をインストール
sudo amazon-linux-extras install -y epel; sudo yum install -y stress

# 600 秒間 CPU 負荷をかける
stress -c 1 -t 600
```


5-4. CloudWatch Logsを確認する

1. ロードバランサーのDNS名を利用してブログサイト (WordPress) へのアクセスを複数回行います
2. AWS マネジメントコンソールで [サービス] から [CloudWatch] をクリックします。
3. 画面左側のメニューから [ロググループ] をクリックします。
4. CloudWatchのロググループに以下のログストリームが作成されていることを確認します。
 - HttpAccessLog
 - HttpErrorLog
5. 「HttpAccessLog」をクリックし、「i-xxxx」から始まるログストリーム名をクリックすると出力されているログの内容を確認することができます。
6. 最後にログの分析を行うために、画面左側のメニューから [ログのインサイト] をクリックします。
7. 以下の指定を行い、「クエリの実行」をクリックし、クエリの実行結果を確認します。

設定項目	設定値
以下によってロググループを選択	ロググループ名
選択基準	HttpAccessLog
<pre>parse '* - * [*] "*" * * * *' as host, identity, dateTimeString, httpVerb, url, protocol, statusCode, bytes,Referer,UserAgent stats count(*) as requestNum by url sort requestNum desc limit 10</pre>	

※ このLogsInsightsクエリは、Webサーバーのアクセスログから最もアクセス数の多いURL上位10件を取得するクエリです。

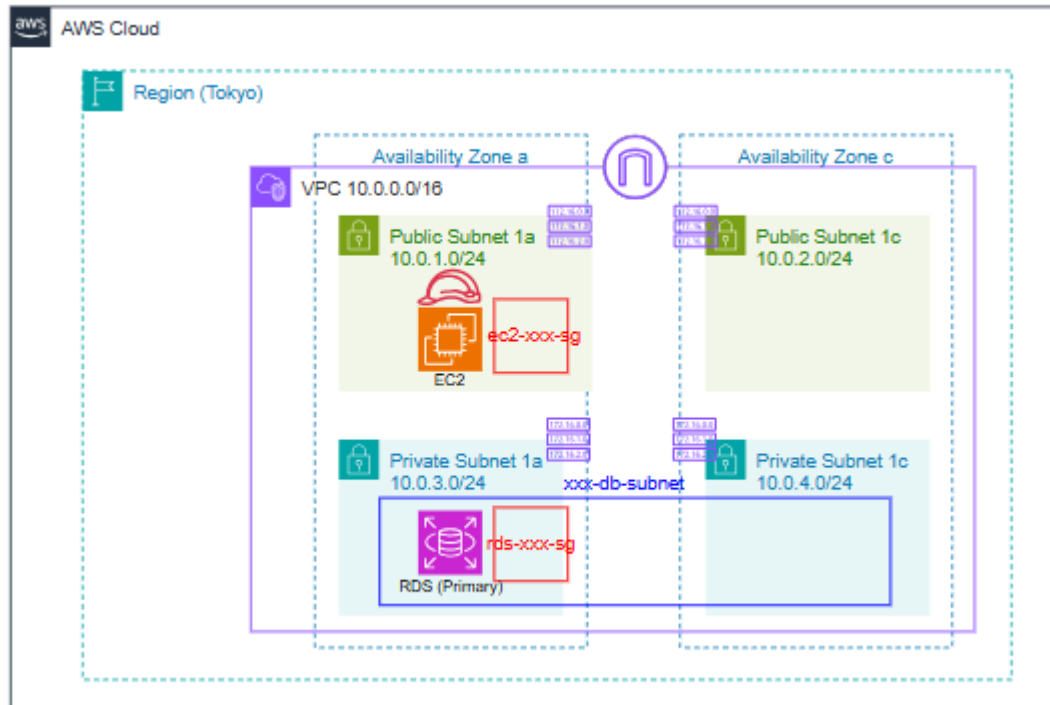
- parse: ログの各行を項目別に分割
- stats: URL別にアクセス数をカウント

- sort: アクセス数の多い順に並び替え
- limit: 上位10件だけ表示

タスク6. IaCによるリソースの作成

タスクのGOAL

- ✓ CloudFormationテンプレートおよびTeraformを利用してリソースを作成する
- ✓ サービスを起動し、確認を行った後は、必ず都度サービスを停止させる
- ✓ タスク完了後のアーキテクチャは以下の通りです



タスクの流れ

以下の流れをCloudFormationとTeratermでそれぞれ実施します。

- 6-1. ネットワークレイヤーを記述する
- 6-2. アプリケーションレイヤーを記述する
- 6-3. データベースレイヤーを記述する

タスクの実施手順 (CloudFormationの場合)

以下の手順を、ネットワークレイヤー、アプリケーションレイヤー、データベースレイヤーに分けて3回実施します。

1. AWS マネジメントコンソールで [サービス] から [CloudFormation] をクリックします。
2. 「スタックの作成」をクリックします。
3. スタックの作成画面で以下の値を指定して「次へ」をクリックします。

設定項目	設定値
テンプレートの準備	既存のテンプレートを選択
テンプレートソース	テンプレートファイルのアップロード
テンプレートファイルのアップロード	「ファイルの選択」をクリック > 「network.yaml」をアップロード

4. スタックの詳細を指定画面で以下の値を指定して「次へ」をクリックします。

設定項目	設定値
スタック名	Network-Stack

※ パラメータが存在する場合はデフォルトのままにしてください

5. スタックオプションの設定画面では何も指定を行わずに「次へ」をクリックします。

※ 「application.yaml」を指定した場合は、画面最下部に「AWS CloudFormation によって IAM リソースがカスタム名で作成される場合があることを承認します。」というメッセージが表示されるのでチェックをつけます。

6. 確認して作成画面では、設定内容を確認して「送信」をクリックします。
7. スタック画面で作成したスタックのステータスが「CREATE_COMPLETE」になるまで待って、リソースタブや出力タブ、実際に作成されたリソースを確認します。

ネットワークレイヤーのリソース作成が終わったら「application.yaml」と「database.yaml」を利用して同じようにアプリケーションレイヤーとデータベースレイヤーのリソースを作成します。

動作確認

- すべてのレイヤーのリソースを作成したら、EC2のパブリックIPアドレスへHTTPアクセスを行うとnginxのウェルカムページを確認することができます。
- 作成したEC2インスタンスへEC2Instance Connect を利用して接続し、以下のコマンドを実行することでRDSへ接続することができます。

```
# RDSへアクセス

mysql -h RDSのエンドポイント -u admin -p

#password: と表示されたら「password」と入力（入力値は非表示です）
password

# MySQL> と表示されRDSへログインできたらデータベース一覧を表示
show databases;
```

タスクの実施手順 (Terraformの場合)

Terraformの実行する方法は複数存在しますが、今回はAWS Cloudshellを利用した方法を紹介します。

1. AWS マネジメントコンソールで [サービス] から [CloudShell] をクリックします。
2. 「AWS CloudShellへようこそ」と表示されたら内容を確認して「閉じる」をクリックします。
3. 以下のコマンドを順番に実行してTerraformをインストールします。

```
# Terraformをダウンロード
wget https://releases.hashicorp.com/terraform/0.14.2/terraform_0.14.2_linux_amd64.zip

# ダウンロードファイルを解凍
sudo unzip terraform_0.14.2_linux_amd64.zip -d /usr/local/bin/

# Terraformのバージョンを確認
terraform -version

# 作業用ディレクトリを作成
mkdir terraform-project
```

4. 画面右上の「アクション」ボタンをクリックし「ファイルのアップロード」をクリックします。
5. 「main.tf」を選択して「開く」をクリックします。

6. ファイルがアップロードされたら、同様の手順を繰り返してすべてのTerraformファイルをアップロードします。（アップロードが失敗した場合は間隔をあけて再試行してください）
7. 以下のコマンドを実行してアップロードしたTerraformファイルを作業用ディレクトリへ移動します。

```
# ファイルがアップロードされたディレクトリへ移動
cd /home/cloudshell-user

# TFファイルをまとめて作業ディレクトリへ移動
mv *.tf terraform-project/
```

8. 以下のコマンドを実行して、Terraformファイルを実行します。

```
# 作業ディレクトリへ移動
cd terraform-project/

# Terraformの初期化
terraform init

# プランの確認
terraform plan

# 実行
terraform apply
```

動作確認

- リソースの作成が完了したら作成されたリソースを確認します
- 作成したら、EC2のパブリックIPアドレスへHTTPアクセスを行うとnginxのウェルカムページを確認することができます。
- 作成したEC2インスタンスへEC2Instance Connect を利用して接続し、以下のコマンドを実行することでRDSへ接続することができます。

```
# RDSへアクセス

mysql -h RDSのエンドポイント -u admin -p

# password: と表示されたら「password」と入力（入力値は非表示です）
password

# MySQL> と表示されRDSへログインできたらデータベース一覧を表示
show databases;
```

後片付け

以上で演習は終了となります。

演習内で作成したリソースは停止や削除を行い、コストを節約することを徹底してください。

「チェックシート」での確認も併せてご実施ください。